C++17 Parallel Algorithms - Execution Policies and Performance

Theory, Edge Cases & Code Examples

- ✓ Data Races with Parallel Execution
- ✓ Locks in Vectorized Code
- ✓ Algorithm Returns Wrong Result Due to Non-Associative Operat
- ✓ Exception Handling in Parallel Algorithms
- ✓ Performance Degradation with Small Data Sets

→ Swipe to learn →

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 1. Data Races with Parallel Execution

```
#include <algorithm>
#include <execution>
#include <vector>
#include <iostream>

// ❌ DANGEROUS: Race condition
void buggy_parallel() {
    std::vector<int> data(1000);
    std::iota(data.begin(), data.end(), 1);

    int sum = 0;  // Shared mutable state

    std::for_each(std::execution::par, data.begin(), data.end(),
        [&sum](int x) {
            sum += x;  // ❌ Data race! Multiple threads access sum
        });

    std::cout << "Sum: " << sum << "\n";  // Undefined result
}

// ✅ CORRECT: Use reduce for parallel aggregation
void correct_parallel() {
    std::vector<int> data(1000);
    std::iota(data.begin(), data.end(), 1);

    int sum = std::reduce(std::execution::par,
                          data.begin(), data.end(),
                          0);  // Thread-safe reduction

    std::cout << "Sum: " << sum << "\n";  // Correct result: 500500
}
```

Why reduce() is safe:

`reduce` uses temporary per-thread accumulators that are combined at the end, avoiding data races. `for_each` provides no such guarantee.

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 2. Locks in Vectorized Code (1/2)

```
#include <algorithm>
#include <execution>
#include <vector>
#include <mutex>

std::mutex mtx;
std::vector<int> shared_log;

// ❌ WRONG: Lock in par_unseq causes undefined behavior
void buggy_vectorized() {
    std::vector<int> data(1000);

    std::for_each(std::execution::par_unseq, data.begin(), data.end(),
        [&](int x) {
            std::lock_guard<std::mutex> lock(mtx);  // ❌ Lock in vectorized code!
            shared_log.push_back(x);  // Undefined behavior
        });
}

// ✅ CORRECT: Use par (not par_unseq) when locks are needed
void correct_parallel_with_lock() {
    std::vector<int> data(1000);

    std::for_each(std::execution::par, data.begin(), data.end(),
        [&](int x) {
            std::lock_guard<std::mutex> lock(mtx);  // ✅ OK with par
            shared_log.push_back(x);
        });
}

// ✅ BETTER: Avoid locks with thread-local accumulation
void best_approach() {
    std::vector<int> data(1000);

    // Process in parallel without locks
    std::vector<int> filtered;
    std::copy_if(std::execution::par,
                 data.begin(), data.end(),
                 std::back_inserter(filtered),
                 [](int x) { return x % 2 == 0; });
}
```

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 2. Locks in Vectorized Code (2/2)

Key insight: `par_unseq` assumes operations can be interleaved arbitrarily (vectorized), so locks cause undefined behavior.

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 3. Algorithm Returns Wrong Result Due to Non-Associative Operations

```
#include <numeric>
#include <execution>
#include <vector>
#include <iostream>
#include <iomanip>

// Floating-point addition is NOT associative due to rounding
void demonstrate_non_associativity() {
    std::vector<double> data = {1e20, 1.0, -1e20, 1.0};

    // Sequential accumulate
    double seq_sum = std::accumulate(data.begin(), data.end(), 0.0);
    std::cout << "Sequential sum: " << std::fixed << seq_sum << "\n";
    // Output: 2.0 (correct due to ordering)

    // Parallel reduce
    double par_sum = std::reduce(std::execution::par,
                                 data.begin(), data.end(),
                                 0.0);
    std::cout << "Parallel sum: " << std::fixed << par_sum << "\n";
    // Output: May be 0.0 or 2.0 depending on grouping!
    // (1e20 + (-1e20)) + 1.0 + 1.0 = 2.0
    // 1e20 + (1.0 + (-1e20)) + 1.0 = 0.0 (1.0 lost to rounding)
}
```

Lesson: Parallel algorithms may change operation ordering, exposing non-associative behavior. Be careful with floating-point arithmetic.

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 4. Exception Handling in Parallel Algorithms

```
#include <algorithm>
#include <execution>
#include <vector>
#include <iostream>

void exception_handling() {
    std::vector<int> data = {1, 2, 3, 4, 5};

    try {
        std::for_each(std::execution::par, data.begin(), data.end(),
            [](int x) {
                if (x == 3) {
                    throw std::runtime_error("Error processing 3");
                }
                std::cout << x << " ";
            });
    } catch (const std::exception& e) {
        // All parallel work is terminated
        // Exception caught AFTER partial work may have completed
        std::cout << "\nCaught: " << e.what() << "\n";
    }
}
```

Behavior:

- Exception in any thread terminates all parallel work
- `std::terminate()` called if multiple threads throw
- Work may partially complete before exception propagates
- Don't rely on side effects in exception case

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 5. Performance Degradation with Small Data Sets (1/2)

```
#include <algorithm>
#include <execution>
#include <vector>
#include <chrono>
#include <iostream>

using namespace std::chrono;

void benchmark_overhead() {
    const int SIZE = 100;  // Small data set
    std::vector<int> data(SIZE);
    std::iota(data.begin(), data.end(), 1);

    // Sequential
    auto start = high_resolution_clock::now();
    std::sort(data.begin(), data.end());
    auto end = high_resolution_clock::now();
    auto seq_time = duration_cast<microseconds>(end - start);

    // Shuffle for next test
    std::random_shuffle(data.begin(), data.end());

    // Parallel
    start = high_resolution_clock::now();
    std::sort(std::execution::par, data.begin(), data.end());
    end = high_resolution_clock::now();
    auto par_time = duration_cast<microseconds>(end - start);

    std::cout << "Sequential: " << seq_time.count() << "µs\n";
    std::cout << "Parallel:   " << par_time.count() << "µs\n";
    std::cout << "Speedup:    " << (double)seq_time.count() / par_time.count() << "x\n";

    // Typical output for small data:
    // Sequential: 2µs
    // Parallel:   50µs
    // Speedup:    0.04x (25x SLOWER!)
}
```

C++17 Parallel Algorithms - Execution Policies and Performance

Edge Cases

📌 5. Performance Degradation with Small Data Sets (2/2)

Threshold guidance:

- `sort`: Useful above ~10,000 elements
- `transform`: Useful above ~50,000 elements with expensive operations
- `reduce`: Useful above ~100,000 elements
- **Always benchmark** your specific use case

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 2. Parallel Point Cloud Processing (LiDAR) (1/3)

Performance characteristics:

- LiDAR processing: 10-30 ms per frame
- With parallelization: 2-5 ms per frame
- Critical for real-time perception (10-30 Hz update rate)

```
#include <algorithm>
#include <execution>
#include <vector>
#include <cmath>
#include <iostream>

struct Point3D {
    double x, y, z;
    double intensity;
};

// Filter ground points (below certain Z threshold)
bool is_ground_point(const Point3D& p) {
    return p.z < 0.5;  // Below 50cm
}

// Calculate distance from origin
double point_distance(const Point3D& p) {
    return std::sqrt(p.x * p.x + p.y * p.y + p.z * p.z);
}

// Normalize point intensity
Point3D normalize_intensity(const Point3D& p, double max_intensity) {
    return {p.x, p.y, p.z, p.intensity / max_intensity};
}

void process_lidar_scan() {
    // 100,000 point cloud (typical LiDAR frame)
    const int NUM_POINTS = 100'000;
    std::vector<Point3D> cloud(NUM_POINTS);

    // Generate test data
    std::generate(cloud.begin(), cloud.end(),
// ...continues on next slide
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 2. Parallel Point Cloud Processing (LiDAR) (2/3)

```
// ...continued from previous slide
        []() {
            return Point3D{
                rand() % 100 / 10.0 - 5.0,  // x: -5 to 5
                rand() % 100 / 10.0 - 5.0,  // y: -5 to 5
                rand() % 100 / 10.0 - 2.0,  // z: -2 to 8
                rand() % 256                  // intensity: 0-255
            };
        });

    // Step 1: Filter non-ground points (parallel)
    std::vector<Point3D> non_ground;
    std::copy_if(std::execution::par,
                 cloud.begin(), cloud.end(),
                 std::back_inserter(non_ground),
                 [](const Point3D& p) { return !is_ground_point(p); });

    std::cout << "Filtered: " << cloud.size() << " → "
              << non_ground.size() << " points\n";

    // Step 2: Calculate max intensity for normalization
    auto max_it = std::max_element(std::execution::par,
                                   non_ground.begin(), non_ground.end(),
                                   [](const Point3D& a, const Point3D& b) {
                                       return a.intensity < b.intensity;
                                   });
    double max_intensity = max_it->intensity;

    // Step 3: Normalize all intensities (parallel)
    std::transform(std::execution::par,
                   non_ground.begin(), non_ground.end(),
                   non_ground.begin(),
                   [max_intensity](const Point3D& p) {
                       return normalize_intensity(p, max_intensity);
                   });

    // Step 4: Calculate distances (parallel reduce)
    std::vector<double> distances(non_ground.size());
    std::transform(std::execution::par,
                   non_ground.begin(), non_ground.end(),
                   distances.begin(),
// ...continues on next slide
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 2. Parallel Point Cloud Processing (LiDAR) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
                   [](const Point3D& p) { return point_distance(p); });

    double total_distance = std::reduce(std::execution::par,
                                        distances.begin(), distances.end(),
                                        0.0);
    double avg_distance = total_distance / distances.size();

    std::cout << "Average point distance: " << avg_distance << "m\n";
}

int main() {
    process_lidar_scan();
    return 0;
}
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 3. Parallel vs Sequential Comparison (1/3)

```
#include <algorithm>
#include <execution>
#include <numeric>
#include <vector>
#include <chrono>
#include <iostream>
#include <iomanip>

void benchmark_parallel_algorithms() {
    const int SIZE = 10'000'000;  // 10 million elements
    std::vector<int> data(SIZE);
    std::iota(data.begin(), data.end(), 1);

    using namespace std::chrono;

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Processing " << SIZE << " elements\n\n";

    // Test 1: Sort
    {
        auto data_copy = data;
        auto start = high_resolution_clock::now();
        std::sort(data_copy.begin(), data_copy.end());
        auto seq_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

        data_copy = data;
        start = high_resolution_clock::now();
        std::sort(std::execution::par, data_copy.begin(), data_copy.end());
        auto par_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

        std::cout << "Sort:\n";
        std::cout << "  Sequential: " << seq_time.count() << "ms\n";
        std::cout << "  Parallel:   " << par_time.count() << "ms\n";
        std::cout << "  Speedup:    " << (double)seq_time.count() / par_time.count() << "x\n\n";
    }

    // Test 2: Transform (square each element)
    {
        std::vector<int> output(SIZE);
// ...continues on next slide
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 3. Parallel vs Sequential Comparison (2/3)

```
// ...continued from previous slide

        auto start = high_resolution_clock::now();
        std::transform(data.begin(), data.end(), output.begin(),
                       [](int x) { return x * x; });
        auto seq_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

        start = high_resolution_clock::now();
        std::transform(std::execution::par, data.begin(), data.end(), output.begin(),
                       [](int x) { return x * x; });
        auto par_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

        std::cout << "Transform:\n";
        std::cout << "  Sequential: " << seq_time.count() << "ms\n";
        std::cout << "  Parallel:   " << par_time.count() << "ms\n";
        std::cout << "  Speedup:    " << (double)seq_time.count() / par_time.count() << "x\n\n";
    }

    // Test 3: Reduce (sum)
    {
        auto start = high_resolution_clock::now();
        long long sum = std::accumulate(data.begin(), data.end(), 0LL);
        auto seq_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

        start = high_resolution_clock::now();
        long long sum_par = std::reduce(std::execution::par, data.begin(), data.end(), 0LL);
        auto par_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

        std::cout << "Reduce (sum):\n";
        std::cout << "  Sequential: " << seq_time.count() << "ms (sum=" << sum << ")\n";
        std::cout << "  Parallel:   " << par_time.count() << "ms (sum=" << sum_par << ")\n";
        std::cout << "  Speedup:    " << (double)seq_time.count() / par_time.count() << "x\n\n";
    }

    // Test 4: Find
    {
        int target = SIZE / 2;

        auto start = high_resolution_clock::now();
        auto it = std::find(data.begin(), data.end(), target);
        auto seq_time = duration_cast<microseconds>(high_resolution_clock::now() - start);
// ...continues on next slide
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 3. Parallel vs Sequential Comparison (3/3)

[Code continues across slides]

```
// ...continued from previous slide

        start = high_resolution_clock::now();
        auto it_par = std::find(std::execution::par, data.begin(), data.end(), target);
        auto par_time = duration_cast<microseconds>(high_resolution_clock::now() - start);

        std::cout << "Find:\n";
        std::cout << "  Sequential: " << seq_time.count() << "µs\n";
        std::cout << "  Parallel:   " << par_time.count() << "µs\n";
        std::cout << "  Speedup:    " << (double)seq_time.count() / par_time.count() << "x\n\n";
    }
}

int main() {
    benchmark_parallel_algorithms();
    return 0;
}

// Typical output on 8-core CPU:
// Processing 10000000 elements
//
// Sort:
//   Sequential: 1450ms
//   Parallel:   250ms
//   Speedup:    5.80x
//
// Transform:
//   Sequential: 85ms
//   Parallel:   15ms
//   Speedup:    5.67x
//
// Reduce (sum):
//   Sequential: 28ms
//   Parallel:   5ms
//   Speedup:    5.60x
//
// Find:
//   Sequential: 45000µs
//   Parallel:   8500µs
//   Speedup:    5.29x
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 4. Transform-Reduce for Dot Product (1/2)

`transform_reduce` is ideal for map-reduce style operations common in data processing and ML inference.

```
#include <algorithm>
#include <execution>
#include <numeric>
#include <vector>
#include <iostream>

// Efficient parallel dot product
double dot_product_parallel(const std::vector<double>& a,
                            const std::vector<double>& b) {
    return std::transform_reduce(
        std::execution::par,
        a.begin(), a.end(),
        b.begin(),
        0.0,  // Initial value
        std::plus<>(),  // Reduction operation
        std::multiplies<>()  // Transform operation
    );
}

// Sequential equivalent
double dot_product_sequential(const std::vector<double>& a,
                              const std::vector<double>& b) {
    double result = 0.0;
    for (size_t i = 0; i < a.size(); ++i) {
        result += a[i] * b[i];
    }
    return result;
}

void demonstrate_transform_reduce() {
    const int SIZE = 10'000'000;
    std::vector<double> vec1(SIZE, 1.5);
    std::vector<double> vec2(SIZE, 2.0);

    using namespace std::chrono;

    auto start = high_resolution_clock::now();
// ...continues on next slide
```

C++17 Parallel Algorithms - Execution Policies and Performance

Code Examples

📌 4. Transform-Reduce for Dot Product (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    double seq_result = dot_product_sequential(vec1, vec2);
    auto seq_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

    start = high_resolution_clock::now();
    double par_result = dot_product_parallel(vec1, vec2);
    auto par_time = duration_cast<milliseconds>(high_resolution_clock::now() - start);

    std::cout << "Dot product of " << SIZE << " elements:\n";
    std::cout << "Sequential: " << seq_time.count() << "ms (result: " << seq_result << ")\n";
    std::cout << "Parallel:   " << par_time.count() << "ms (result: " << par_result << ")\n";
    std::cout << "Speedup:    " << (double)seq_time.count() / par_time.count() << "x\n";
}

int main() {
    demonstrate_transform_reduce();
    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Data Races with Parallel Execution
- ✓ Locks in Vectorized Code
- ✓ Algorithm Returns Wrong Result Due to Non-Associative Operat
- ✓ Exception Handling in Parallel Algorithms

→ *Swipe to learn* →